Snake Game

Snake Game

Game Function

All the player needs to choose direction and have to feed an apple. Each time snake is being feed, the size of snake grows. And whenever snake’s head hit the boundaries of the wall player is out.

Win/Lose Conditions

WIN
When snake fills the entire grid (may be)
lOSE
When the snakes head touches a wall or any parts of its own body

User Interface (UI)

Screen shows the grid, snake, apple and the current score

Controls (keyboard and mouse)

Arrows keys change the snake’s direction, snake can’t turn 180

Scoring

Player gets +1 point for each apple eaten

Boundaries

Snake can only move inside the grid, touching wall ends the game

Infinite Castle Of Hidden Door

Infinite Castle Of Hidden Door

Game Goal (Added element)

Explore the Infinite Castle and open the hidden doors. Defeat the demon in each door. Beat all 7 worlds, then the mini-boss and the final boss to win.

Game Description & Story (Added element)

It is midnight, and you are a demon slayer inside the Infinite Castle. Each door looks different, and its look tells you which world is inside. Some doors lead back to the castle. You pick one hero at the start, and each world has its own hero. Beat a world to unlock its hero.

Game Function

Run through the castle and press ▲ at a door to open it. Inside a world, a demon hides, then pops up in a random spot for 16 seconds. If it vanishes, it comes back somewhere else. The demon walks toward you and attacks. Hit it with your sword or punches before it escapes. When you win, the door closes. A door also closes after you use it.
Sword: one hit. A miss costs 10 seconds.
Punch: no penalty, but it takes 3 punches to make 1 hit.
Hearts: you have 3 ❤. You lose 1 when a demon hits you.
Levels: heroes level up after 3, 5, and 7 wins (max level 3). Heroes are saved.
Hero Powers (recharge in 10 seconds)
Kagaya: water, +8 seconds
Giyu: fire, burns a demon from far away
Kocho: poison, the demon slowly loses health
Iguro: teleports behind the demon

Kanroji: fastest, hits and runs fast
Amane: invisible for 7 seconds
Gyomei: strength, extra damage

Win/Lose Condition

Win: Beat one demon in each of the 7 worlds. Beat the mini-boss after 3 worlds. Then beat the final boss.
Lose: The timer reaches zero, a demon escapes 2 times, or your hearts run out.

User Interface(UI)

The screen shows the timer, score, doors used, worlds cleared, your hero, and your hearts.

Controls (mouse and keyboards)

◂ ▸ arrows: run
▲ arrow: enter door
Space: sword
▼ arrow or Shift: punch
Right-click: power
On phone: use the on-screen buttons, or tap the screen to use the sword.

Scoring

You get seconds left × 10 points when you beat a demon. Each hero level gives +25%. If your hero matches the world, you get +50% more.

Boundaries

You can only move inside the castle and the world you are in. Each world gives you 90 seconds. Doors close after you use them.

Technical Requirements (Added element)

Use HTML, CSS, and JavaScript. The game must work on GitHub Pages without a server. Put everything in one game_name.html file. This will be stored in my student portfolio re.

Tab 5

GPT chat link to my improved prompt:

https://claude.ai/share/309ef3e7-25d9-446b-a147-1ec602cb9667

Prompt:

You are a game developer.

Program the game designed around my #Game Elements below. The name of my game is CLI Dungeon Crawl.

This is an educational game to help people understand how to use the command line on windows or linux systems.

Use HTML, CSS, and JavaScript. The game must work on GitHub Pages without a server. Put everything in one game_name.html file. This will be stored in my student portfolio repo and linked to my portfolio site.

#Game Elements

Game Goal (Added element)

I want to make a game that helps users understand the command line interface as part of a technical course in building computer usage skills where players have to learn commands to navigate a virtual filesystem to complete a goal.

Game Description & Story (Added element)

You are an adventurer that has recently been tasked with a quest to find a hidden treasure in the dungeons of a maniacal wizard's fortress. You seek your fame and fortune by accepting the quest in full awareness that you are not the first to try. Death or capture likely awaits you.

Game Function

All play is conducted through a simulated CLI or terminal styled interface where the user issues commands to advance and act on the game world.. There should be no graphical visuals other than the normal text and window elements of a terminal. There should be a virtual file structure that acts as the rooms (folders/directories), areas (folders/directories), objects (files), and NPCs (files) in the wizard's fortress and dungeons below.

Win/Lose Conditions

Winning should require the player to play to a successful end-game state. There may be several end-game states but the first should be finding the treasure.

Losing should happen if the player is trapped in a room, found by the wizard, or killed in a trap.

User Interface (UI)

Terminal window as a centered hovering window. There should be a score board above it somewhere but not too distracting from the main terminal window. The Windows and Linux terminal window should look distinct stylistically to signify to the player which one they are using.

Controls (keyboard and mouse)

Physical controls:
Keyboard mostly. There could be hidden elements within the game screen or UI that could allow the user to use the mouse. I haven't considered what exactly yet so for now the mouse does nothing.

Game controls:
The game is controlled through commands issued by the user. The available commands can always be seen by typing the help command. Maybe new commands can be found and added by the player's exploration. Command should mirror the normal windows or linux commands. There should be an option at the beginning of the game that asks the user for their learning preference between windows or linux terminal. Commands should act as required by the game. This means they should be clearly defined to the player and may be able to be altered through game actions.

Scoring

Scores should be conducted based on actions. I am reminded of Kings Quest III where you could beat the game but have varying scores since there were multiple ways to do the same action. For simplicity, each room uncovered is worth 1 point, objects are tiered by importance (this can be a single to the player) between 1 and 4 points, actions using objects are worth 2 points, and getting the treasure is worth 10.

Boundaries

The virtual folder structure is the internal boundary and the terminal window the playable, interactive area of the game. However, the player may require paper notes or spreadsheets to keep track of the folder structure. This extends play to the real physical world.

Technical Requirements

Use HTML, CSS, and JavaScript. The game must work on GitHub Pages without a server. Put everything in one game_name.html file. This will be stored in my student portfolio repo and linked to my portfolio site.

Tab 6

Help me improve the clarity and specificity of my game-building prompt.

Your role: You are a prompt editor. I am the game designer. Preserve my main idea and help me make it clear enough to build.

Assignment rules: The game must be original. No clones, simple picture matching, or memory card games. Inspiration is allowed, but the gameplay must have meaningful differences, player decisions, and connected rules.

Follow this process:

1. Review my prompt for: function and goal, win condition, lose condition, controls, boundaries, user interface, scoring, and connected rules.
2. Identify the most important missing details, unclear rules, and contradictions. Prioritize problems that could change how the game plays or prevent it from meeting the assignment rules.
3. Ask **no more than 10 follow-up questions in total**, one at a time. Each question should address one main decision. Do not hide extra questions inside it.
4. For each question:
    - Give **2–3 specific answer options** that fit my game.
    - Mark the best-fit option **Recommended** and briefly explain why.
    - Include **"My own answer"** and **"Let AI decide."**
    - Use simple English and short sentences.
    - Label it **Question 1 of 10**, **Question 2 of 10**, and so on.
5. Wait for my answer before continuing. Adapt your next question to my answers. Stop early if the major concerns are resolved; you do not need to use all 10 questions.
6. After the questions, fill in any remaining details using reasonable choices that fit my idea and answers. You may contribute supporting rules, numerical values, layout, visual details, and technical solutions. Do not replace my main game concept or contradict my decisions.
7. If my idea does not meet the originality requirements, address that within the question limit. Help me develop my existing idea rather than replacing it.

Final output:

- Write a complete game-building prompt with clear sections, specific rules, numbers, and consequences.
- Include both my decisions and the supporting details you contributed.
- After the prompt, briefly list **AI-added decisions** so I can identify and review them.
- If a major assignment requirement remains unmet, state it clearly instead of claiming the prompt meets every requirement.

Include this technical requirement in the final prompt:
"Use HTML, CSS, and JavaScript. The game must work on GitHub Pages without a server. Put everything in one index.html file."

Do not generate game code yet.

My game prompt: